

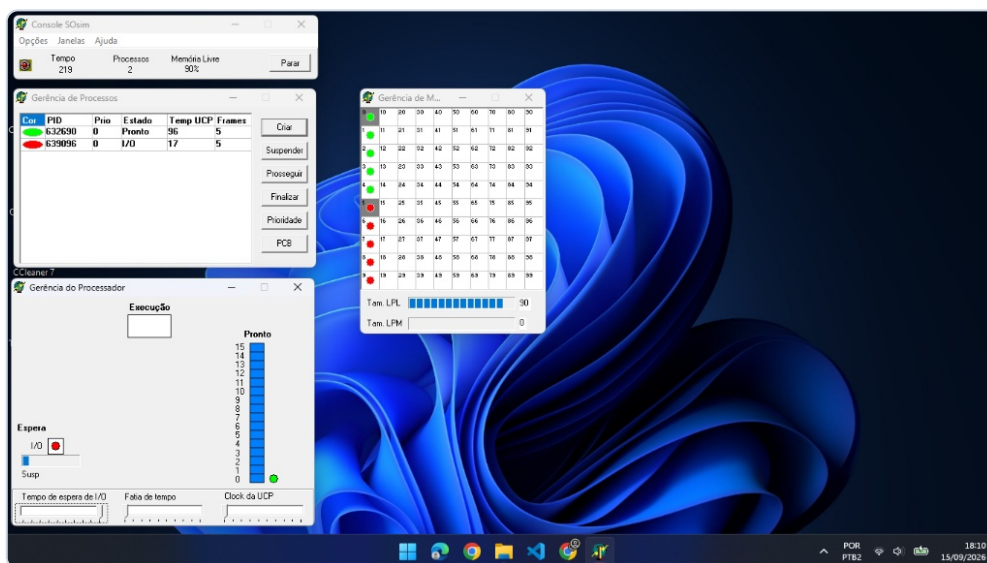
Relatório de Laboratório — SOsim

Gerência do Processador — Exercícios de Escalonamento

Exercício 1: Escalonamento Circular (Round-Robin)

1. Configuração do Experimento

- **Algoritmo de Escalonamento:** Circular (Round-Robin / Sem Prioridade)
- **Criação de Processos:** 2 processos com a mesma prioridade (Prioridade 0)
 - **Processo Verde (PID 632690):** CPU-bound (alta demanda de uso do processador).
 - **Processo Vermelho (PID 639096):** I/O-bound (alta frequência de operações de entrada/saída).
- **Tempo de Observação:** 3 minutos (Tempo final no console: 219)



captura de tela após 3 minutos

2. Observações e Mudanças de Estado

- **Processo CPU-bound (Verde - PID 632690):** Alterna constantemente entre **Pronto (Ready)** e **Executando (Running)**. Utiliza todo o *time slice* (quantum) concedido e sofre preempção ao final do tempo. **Tempo UCP: 96.**
- **Processo I/O-bound (Vermelho - PID 639096):** Alterna frequentemente entre **Executando (Running)**, **Espera I/O (Waiting/Blocked)** e **Pronto (Ready)**. Abandona voluntariamente a UCP no início do quantum ao solicitar I/O. **Tempo UCP: 17.**

3. Análise da Distribuição do Uso da UCP

A UCP é fortemente dominada pelo processo verde (CPU-bound) com 96 unidades de tempo versus 17 do processo vermelho. Como o processo I/O-bound se bloqueia voluntariamente antes de esgotar o quantum, o processo CPU-bound aproveita quase todo o tempo de processamento restante na fila.

Pergunta: O que acontece se o tempo de time slice (quantum) aumentar ou diminuir?

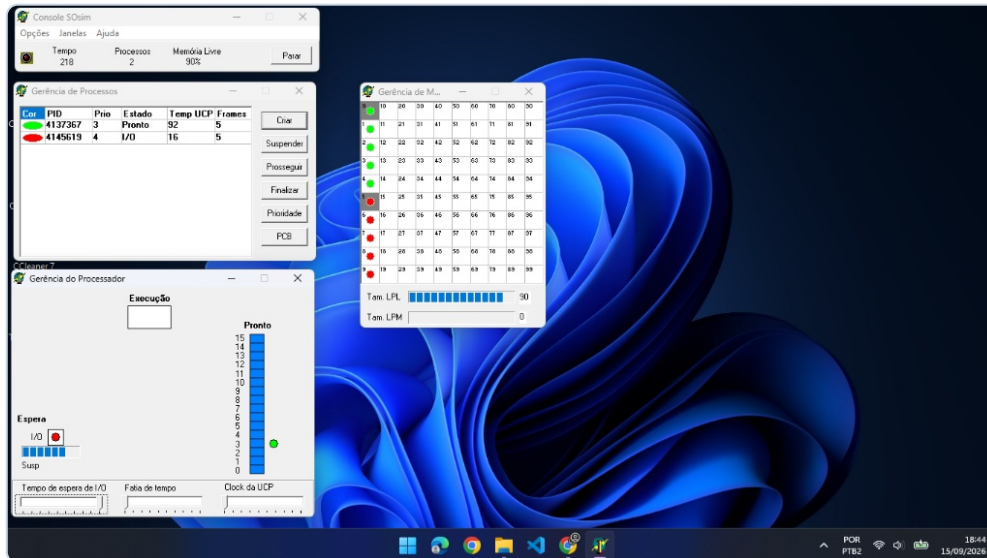
Se Aumentar: O sistema se aproxima do algoritmo FIFO/FCFS. O processo CPU-bound ganha mais desempenho por ter menos trocas de contexto (overhead), porém a responsividade e o tempo de resposta do processo I/O-bound pioram.

Se Diminuir: Aumenta a sensação de simultaneidade e melhora a responsividade para o processo I/O-bound, mas eleva drasticamente o overhead por excesso de trocas de contexto, reduzindo a eficiência geral da UCP.

Exercício 2: Escalonamento Circular com Prioridades

1. Configuração do Experimento

- **Algoritmo de Escalonamento:** Prioridades (Com Prioridade)
- **Criação de Processos:**
 - **Processo Verde (PID 4137367):** CPU-bound com Prioridade 3.
 - **Processo Vermelho (PID 4145619):** I/O-bound com Prioridade 4.
- **Tempo de Observação:** 3 minutos (Tempo final no console: 218)



captura de tela após 3 minutos

2. Observações e Mudanças de Estado

- **Processo CPU-bound (Verde - PID 4137367 / Prio 3):** Alterna entre **Pronto** e **Execução**, mantendo prioridade de execução na UCP. **Tempo UCP: 92.**
- **Processo I/O-bound (Vermelho - PID 4145619 / Prio 4):** Alterna entre **Execução**, **Espera I/O** e **Pronto**. Por ter prioridade menor e realizar pausas de E/S, executa menos vezes. **Tempo UCP: 16.**

3. Análise Comparativa ao Exercício 1

A atribuição de prioridades acentua o domínio do processo CPU-bound (Verde). Sempre que o processo Verde está na fila de prontos, o escalonador concede preferência a ele em relação ao processo Vermelho de menor prioridade.

Pergunta: O que acontece se o tempo de espera do processo I/O-bound aumentar ou diminuir?

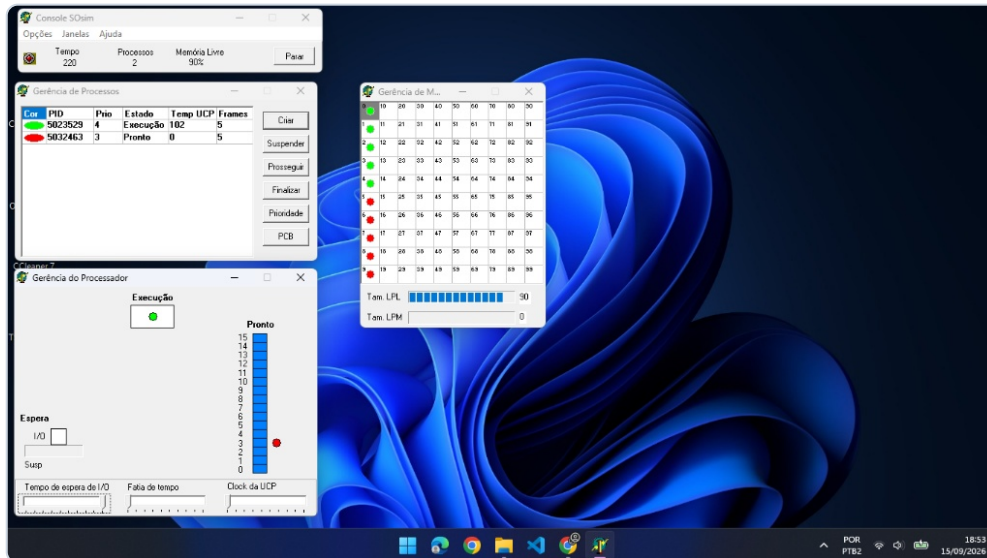
Se Aumentar: O processo I/O-bound passará mais tempo no estado *Bloqueado/Espera I/O*, liberando ainda mais tempo de processamento para o processo CPU-bound e reduzindo seu tempo total acumulado de UCP.

Se Diminuir: O processo I/O-bound retornará para a fila de *Prontos* mais rapidamente. Se possuir prioridade suficiente para executar, aumentará sua frequência de uso da UCP para disparar novas chamadas de E/S.

Exercício 3: Escalonamento Circular com Prioridades (Inanição / Starvation)

1. Configuração do Experimento

- **Algoritmo de Escalonamento:** Prioridades
- **Criação de Processos:**
 - **Processo Verde (PID 5023529):** CPU-bound com Prioridade 4 (Prioridade Maior).
 - **Processo Vermelho (PID 502463):** I/O-bound com Prioridade 3 (Prioridade Menor).
- **Tempo de Observação:** 3 minutos (Tempo final no console: 220)



captura de tela após 3 minutos

2. Observações e Análise da Situação

- **Processo Verde (CPU-bound / Prio 4):** Permanece continuamente no estado **Execução (Running)**. Monopolizou 100% da UCP. **Tempo UCP: 102.**
- **Processo Vermelho (I/O-bound / Prio 3):** Permanece retido no estado **Pronto (Ready)** sem conseguir executar uma única vez. **Tempo UCP: 0.**

Fenômeno Observado: Ocorreu *Starvation (Inanição)*. O processo CPU-bound de maior prioridade nunca solta a UCP, impedindo que o processo I/O-bound execute até mesmo sua primeira instrução para solicitar E/S.

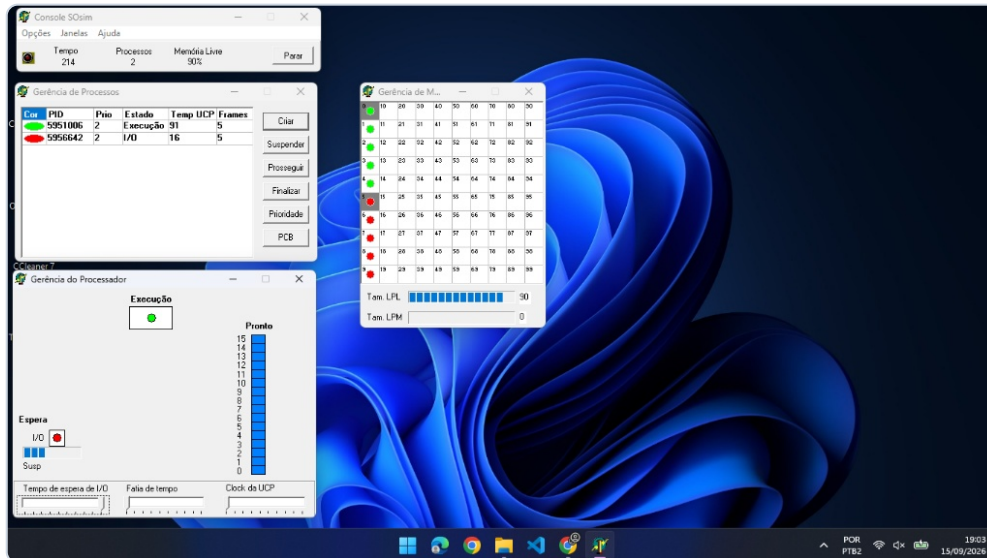
Pergunta: Quais devem ser os critérios para determinar as prioridades de processos?

1. **Favorecer Processos I/O-bound e Interativos:** Conceder prioridade mais alta para processos I/O-bound garante alta responsividade do sistema, pois eles utilizam a UCP brevemente e se bloqueiam em seguida.
2. **Mecanismo de Envelhecimento (Aging):** Aumentar dinamicamente a prioridade de processos que esperam há muito tempo na fila de *Prontos* para evitar a inanição (starvation).
3. **Importância/Crítica do Sistema:** Processos essenciais do Kernel devem possuir maior prioridade em relação às tarefas em lote (batch).

Exercício 4: Escalonamento com Prioridade Dinâmica (Mecanismo Adaptativo)

1. Configuração do Experimento

- **Algoritmo de Escalonamento:** Prioridades (Empate de Prioridade)
- **Criação de Processos:** 2 processos com a mesma prioridade (Prioridade 2)
 - **Processo Verde (PID 5951006):** CPU-bound com Prioridade 2.
 - **Processo Vermelho (PID 5956642):** I/O-bound com Prioridade 2.
- **Tempo de Observação:** 3 minutos (Tempo final no console: 214)



captura de tela após 3 minutos

2. Observações e Comparação com o Exercício 2

- **Processo Verde (CPU-bound):** Alterna entre Pronto e Execução, aproveitando a fatia circular do seu nível. **Tempo UCP: 91.**
- **Processo Vermelho (I/O-bound):** Alterna entre Execução, Espera I/O e Pronto. **Tempo UCP: 16.**

Em caso de empate de prioridades, o sistema utiliza o tempo compartilhado (Round-Robin) para o desempate. Diferente do Exercício 3, o processo I/O-bound consegue executar perfeitamente e realizar suas operações de E/S.

Pergunta: Qual a vantagem desse escalonamento em processos I/O-bound de perfis diferentes?

Maximização do Paralelismo de E/S: Como os processos I/O-bound possuem perfis diferentes (ex: acesso a disco, rede, teclado), o escalonamento permite disparar rapidamente as chamadas de cada um, mantendo múltiplos periféricos operando em paralelo.

Responsividade e Baixa Latência: Como processos I/O-bound não consomem o quantum inteiro, o escalonador alterna entre eles rapidamente sem sobrecarregar a UCP.